GUIA TÉCNICO | 17

Interpretabilidade visual e detecção de objetos

Capítulo 17/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Entender saliência, mapas de ativação e distinguir classificação de detecção.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Mapas de saliência mostram sensibilidade local e não causalidade. Detecção de objetos combina classe e localização, frequentemente por caixas delimitadoras; segmentação estima regiões por pixel. Avaliação exige IoU, precisão e recall por limiar.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: IoU=área(A∩B)/área(A∪B)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Mapas de saliência localizam sensibilidades de predição, mas não estabelecem relação causal; separar interpretação de detecção.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
def iou(a,b):
 x1=max(a[0],b[0]);y1=max(a[1],b[1]);x2=min(a[2],b[2]);y2=min(a[3],b[3])
 inter=max(0,x2-x1)*max(0,y2-y1)
 area=lambda q:(q[2]-q[0])*(q[3]-q[1])
 return inter/(area(a)+area(b)-inter)
print('IoU:',round(iou((0,0,2,2),(1,1,3,3)),3))
assert abs(iou((0,0,2,2),(1,1,3,3))-1/7)<1e-9
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um

conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Mapa de saliência numérica com gradientes de entrada

Mapas de saliência localizam sensibilidades de predição, mas não estabelecem relação causal; separar interpretação de detecção.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
img=np.array([[.1,.2,.3],[.4,.5,.6],[.7,.8,.9]],dtype=float)
weights=np.array([[0.,1.,0.],[0.,2.,0.],[0.,1.,0.]])
def score(x):return float(np.sum(x*weights))
h=1e-5
saliency=np.zeros_like(img)
for i in range(img.shape[0]):
 for j in range(img.shape[1]):
    p=img.copy();q=img.copy();p[i,j]+=h;q[i,j]-=h
    saliency[i,j]=(score(p)-score(q))/(2*h)
assert np.allclose(saliency,weights,atol=1e-8)
print('Saliência numérica:\n',saliency)
print('Saliência é sensibilidade local, NÃO uma explicação causal.')
```

Para um modelo real use gradientes da pontuação da classe em relação à imagem, comparando com abordagens e perturbações alternativas.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Uma heatmap de saliência prova causalidade?

Resposta: Não; destaca regiões sensíveis ao modelo, não necessariamente evidência causal.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide